Guida al sistema

Come è fatta la casa editrice del repository ZeusBot: chi fa cosa, il percorso di un libro, le regole.

Per Felice, autore. Ottobre 2026.

Fotografia del ramo `motore` al commit `138db51`.

> **Come leggere.** Tra parentesi, in grigio, c'è il file da cui viene ogni affermazione. Quando una cosa non c'è nel repository, la guida lo scrive: «non esiste».

Indice

0. Le parole da sapere p. 3
1. A cosa serve il sistema p. 6
2. Organigramma: chi fa cosa p. 8
3. Il percorso di un libro p. 23
4. Le regole p. 40
5. I rami p. 48
6. Cosa non esiste p. 51
7. Appendici e indice p. 54
8. I 17 punti deboli p. 58
9. Dove trovare le cose p. 63

0. Le parole da sapere

Poche parole tecniche tornano spesso. Eccole, una volta per tutte.

- **Repository**: l'archivio del progetto su GitHub, con tutta la storia delle modifiche. Qui si chiama ZeusBot.
- **Ramo**: una versione parallela del repository. Ogni ramo ha i suoi file. Il ramo `motore` contiene la macchina; ogni libro vive su un ramo suo.
- **Commit**: un salvataggio con nome e data. Ha un codice breve, per esempio `138db51`.
- **Push**: l'invio dei salvataggi su GitHub (su «origin», la copia in rete). Finché non c'è il push, il lavoro esiste solo nel computer di lavoro.
- **Merge**: l'unione di un ramo dentro un altro.
- **Script**: un piccolo programma che fa un lavoro preciso (contare parole, fare il PDF…).
- **Hook**: un guardiano automatico. Claude Code lo chiama prima di certe

azioni; l'hook può lasciar passare, avvisare o bloccare.
- **Claude Code**: l'assistente che lavora nel repository, legge le regole e lancia i comandi.
- **YAML**: un formato di testo per i dati, fatto di righe «nome: valore». I file finiscono in `.yaml`.
- **Gate** (fermata): una fermata obbligata. Il lavoro si ferma finché l'autore non approva. Si apre con `zb pronto`.
- **Struttura**: la prima fase dopo il briefing. Prima della prosa si fissano premessa, personaggi, atti, scaletta, ritmo e scelte fisse (punto di vista, tempo, voce).
- **Gate struttura**: la fermata che chiude la fase struttura. È sempre attivo: senza il tuo ok non nasce nessun documento e nessun capitolo.
- **Cartella unica**: tutto ciò che riguarda un libro vive in `libri/<slug>/`, e il nome del libro non compare altrove.
- **Ruolo**: un mestiere editoriale (direttore, architetto, scrittore, editor,

correttori). Una sessione di Claude Code svolge un ruolo solo.

- **Subagente** (o agente): un ruolo che Claude Code chiama come aiutante, con un contesto suo e pulito. Restituisce al massimo 10 righe.
- **Capsula**: il riassunto di continuità di un capitolo, al massimo 180 parole, in `07-capsule/NN.md`. Chi scrive legge le capsule, non i capitoli interi.
- **Audit anti-IA**: il controllo che segnala i segni di scrittura da IA. Il motore segnala e propone; non riscrive.
- **sha256**: l'«impronta digitale» di un file. Se cambia anche una virgola, l'impronta cambia.
- **KDP**: Kindle Direct Publishing, il servizio di Amazon per pubblicare.
- **Metadati del PDF**: dati nascosti nel file, come Title (titolo) e Author (autore).
- **Slug**: il nome breve di un libro usato nei percorsi, senza spazi né accenti.

(motore/editoriale/glossario.md; motore/editoriale/risparmio-token.md)

1. A cosa serve il sistema

Il sistema è una fabbrica di libri, anzi una casa editrice. Serve per romanzi di qualsiasi genere (motore/README.md).

Il cuore è il **motore**: procedura, script, modelli vuoti, profili di genere, dati KDP e il livello editoriale (percorso, ruoli, agenti, anti-IA, correttore). Il motore non contiene libri veri (motore/README.md).

Il percorso è sempre lo stesso. L'autore scrive un briefing. Il motore crea la cartella del libro. Prima si fa la struttura, poi i documenti, poi i capitoli, uno alla volta. Seguono le revisioni, la chiusura meccanica, il PDF di stampa e il pacchetto per KDP. A ogni tappa importante l'autore approva (motore/editoriale/00-percorso.md).

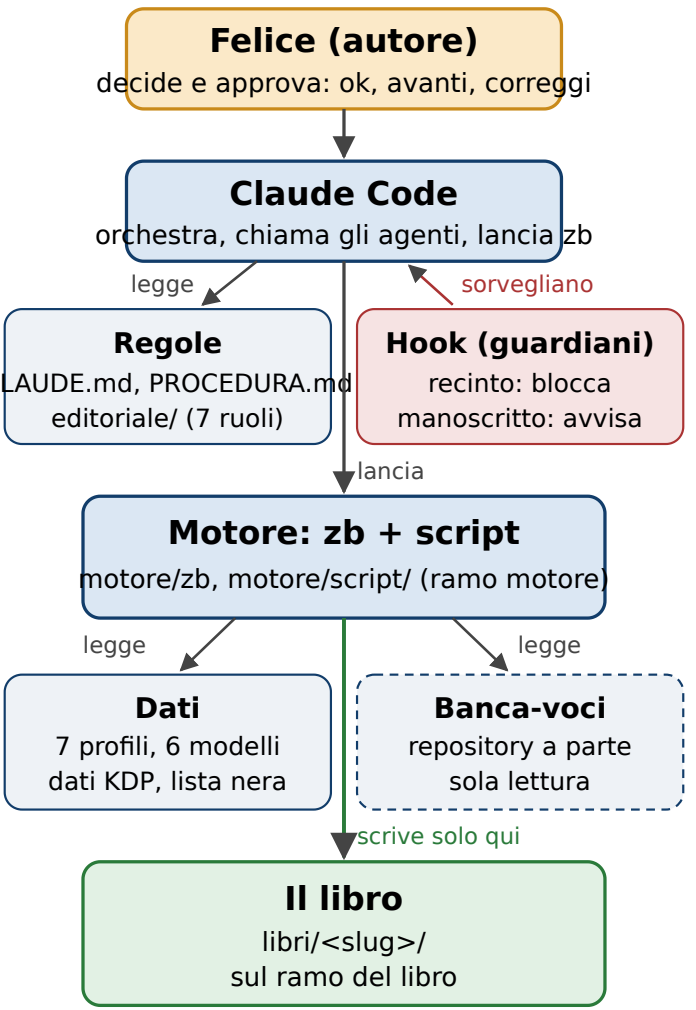
Tre idee reggono tutto (CLAUDE.md):

1. **Il motore resta pulito.** Il ramo `motore` non contiene mai un libro.

2. **Il libro vive solo nella sua cartella.** Mai nel motore, mai altrove.
3. **Proponi, l'autore approva, poi applica.** Claude Code non approva mai da solo.

2. Organigramma: chi fa cosa

Lo schema



Schema ricavato da CLAUDE.md, motore/README.md e motore/editoriale/LEGGIMI.md.

Chi fa cosa, in breve

Chi	Che cosa fa	Dove sta
Felice	Scrive il briefing, risponde ok / avanti / correggi, decide	—
Sessione principale (Claude Code)	Orchestra: chiama l'agente del passo, legge la sua risposta breve, lancia `zb`. **Non scrive prosa**	—
Ruoli e agenti	Fanno il lavoro editoriale	`motore/editoriale/`

Chi	Che cosa fa	Dove sta
Motore	Crea, controlla, compone, impagina	`motore/`
Regole	Dicono come si lavora	`CLAUDE.md`, `motore/PROCEDURA.md`, `motore/editoriale/`
Hook	Sorvegliano le scritture	`motore/script/hook_*.py`
Libro	Il lavoro vero	`libri/<slug>/`

(CLAUDE.md; motore/README.md; motore/editoriale/risparmio-token.md)

I sette ruoli editoriali

Ogni ruolo ha un file in `motore/editoriale/ruoli/`. Legge solo i file del suo «Input» e scrive un report in `06-diagnostica/` del libro. Non modifica il manoscritto, salvo la correzione, e solo

dopo il tuo ok (motore/editoriale/LEGGIMI.md).

#	Ruolo	Lavoro	Fermata
1	Direttore editoriale	identità e promessa del libro; bibbia, cronologia, nota di direzione	`documenti`
2	Architetto della struttura	atti, svolte, scaletta, sottotrame, ritmo	`struttura`

#	Ruolo	Lavoro	Fermata
3	Scrittore con voce	pagina campione e capitoli, con voce e manuale di stile	`pagina_campione`, `primi_capitoli`
4	Editor strutturale	giudica la struttura e poi il manoscritto intero	nel gate `struttura`; `struttura_manoscritto`
5	Line editor	frase per frase: ritmo, voce, tic da IA	`line`

#	Ruolo	Lavoro	Fermata
6	Correttore di italiano	grammatica, accenti, punteggiatura, coerenza	`copy`
7	Correttore di bozze	ultimo controllo sul libro impaginato	`bozze`

(motore/editoriale/ruoli/; motore/editoriale/00-percorso.md)

Gli agenti

Ogni ruolo ha il suo **subagente** in `motore/editoriale/agenti/`, più due: il **lettore cieco** (passo opzionale) e la **capsula**. In tutto nove (motore/dati/modelli-agenti.yaml).

- **Modelli.** Forte (opus) per architetto ed editor strutturale; medio (sonnet) per direttore, scrittore, line editor,

correttori e lettore cieco; leggero (haiku) per la capsula. La scelta sta in `motore/dati/modelli-agenti.yaml`.

- **In sequenza:** lo scrittore, un capitolo per volta. **In parallelo:** line editor, correttore di italiano e capsula, al massimo 3 alla volta (se arrivano i limiti d'uso, 2 poi 1).
- **Il lettore cieco** legge un campione da lettore, in sola lettura. Solo con il tuo ok.
- **La capsula** riassume un capitolo accettato in massimo 180 parole.
- Gli agenti non fanno commit né push: li fa la sessione principale con `zb push`.
- `zb agenti attiva --ok` li copia in `.claude/agents/`, solo sui rami dei libri.

(motore/dati/orchestrazione.yaml; motore/editoriale/risparmio-token.md; motore/editoriale/agenti/)

Il motore: `zb` e gli script

`zb` è il comando unico. Si scrive `python3 motore/zb <comando> <libro>`. Il libro si indica sempre con il percorso della sua cartella. L'elenco dei comandi è uno solo, nel codice: `python3 motore/zb aiuto` (motore/README.md, «Comandi»).

Gli script sono 34 file in `motore/script/`. Ogni comando stampa al massimo 40 righe e rimanda a un file per il resto; le righe con KO o FERMO restano sempre a schermo (motore/dati/output.yaml).

I comandi di oggi (zb aiuto):

- `zb nuovo`, `zb adotta`: crea il libro dal briefing (fase struttura); prepara le bozze dei dati per un libro già scritto.
- `zb avvio` (con `--ruolo`), `zb ruoli`, `zb cartella`, `zb stato`: controlli di inizio sessione, ruoli, cartella unica, stato. Non scrivono testi.
- `zb pronto`, `zb avanti`, `zb ok`, `zb correggi`, `zb revisione`, `zb`

`riapprova`, `zb fase`: i gate e le risposte dell'autore.

- `zb esito`: i controlli di un capitolo. Dentro ci sono i conteggi, lo stile, il riciclo e l'audit anti-IA.
- `zb capsula`: la capsula di continuità, dopo ogni capitolo.
- `zb continuita`, `zb ortografia`, `zb compila`, `zb impagina`, `zb pdf`, `zb pacchetto`, `zb kdp`: la chiusura meccanica.
- `zb verifica KDP`: registra o ricorda la verifica dei dati KDP.
- `zb agenti`, `zb consumo`, `zb hook`, `zb lavoro`, `zb voce`: agenti, misura delle letture, hook, recinto, banca-voci.
- `zb push`: il push, con le prove sul motore (capitolo 4).
- `zb prove`, `zb separazione`, `zb recinto`, `zb profili`: le prove del motore.

I comandi `pulizia`, `conta`, `stile`, `riciclo` e `capitolo` **non esistono più** come comandi: `conta`, `stile`, `riciclo` e `capitolo` sono diventati parti di `zb`

`esito`, `pulizia` è stato tolto (motore/passaggio-di-consegne.md, «Decisioni prese»).

Nessun comando del motore fa commit da solo (motore/PROCEDURA.md §8).

La «Bibbia editor»

Il nome «Bibbia editor» indicava l'insieme delle regole editoriali. Oggi quell'insieme sta in due posti (motore/README.md; motore/editoriale/LEGGIMI.md):

- `motore/editoriale/`: percorso, ruoli, agenti, struttura, anti-IA, correttore, glossario;
- `motore/PROCEDURA.md`: la procedura completa.

Le aiutano alcuni file di dati, per esempio la lista nera di base (`motore/dati/lista-nera.yaml`) e le parole italiane ammesse (`motore/dati/parole-ammesse-it.txt`).

Precedenza: una sola scala, in `motore/editoriale/LEGGIMI.md` (dall'alto,

vince la prima):

1. manuale del libro e `libro.yaml`
2. briefing e scheda della voce (se contrastano vince il briefing; il contrasto si segnala)
3. profilo di genere
4. ruoli editoriali
5. procedura del motore

Ogni contrasto si segnala (motore/editoriale/LEGGIMI.md; CLAUDE.md).

> Attenzione: la `bibbia.md` di ogni libro (personaggi, luoghi, fatti) è un'altra cosa. Sta nella cartella del libro, in `02-bibbia/`.

Profili di genere

Un profilo dà i valori predefiniti di un genere: lunghezza, capitoli, scene, frase media, dialogo e soglie anti-IA (motore/PROCEDURA.md §3 e §6).

Sono 7, nella cartella `motore/profili/`: fantascienza, fantasy, giallo, narrativa letteraria, romance, thriller, young adult.

Un valore del briefing fuori dal profilo è accettato solo con un «Motivo override:» scritto nel briefing. Senza motivo, `zb nuovo` si ferma (motore/PROCEDURA.md §3).

Voci narranti

Una voce dice **come** si racconta. Il **cosa** lo decide l'autore nel briefing (motore/README.md, «Banca-voci»).

Le voci stanno in un repository separato, la banca-voci, mai dentro il motore. Il motore le legge in sola lettura dalla cartella indicata da `ZB_BANCA_VOCI`. L'elenco è nel suo `indice.yaml`; questa guida non nomina le voci, perché alcune hanno nomi reali (motore/passaggio-di-consegne.md, «Stato»).

Un libro sceglie la voce nel briefing; `zb nuovo` la scrive in `libro.yaml`. `zb voce lista`, `zb voce mostra <slug>` e `zb voce`

`verifica <slug>` le leggono. Se il briefing contrasta con la voce, vince il briefing e il contrasto si segnala (motore/editoriale/LEGGIMI.md).

Modelli

I modelli sono i file vuoti da cui nasce un libro. Stanno in `motore/modelli/` e sono 6: `briefing.md` (il modulo dell'autore), `libro.yaml` (le regole del libro), `stato.yaml` (a che punto è), `cronologia.yaml` (date ed eventi), `LEGGIMI.md` (cosa leggere a ogni avvio), `conferme-autore.yaml` (le conferme per KDP). I modelli dei documenti della struttura stanno a parte, in `motore/editoriale/struttura/modelli/` (cartelle motore/modelli/ e motore/editoriale/struttura/).

Hook

Gli hook sono quattro, nel modello `motore/dati/hook-settings.esempio.json`. Gli effetti sono

nel capitolo 4 (motore/README.md, «Hook di Claude Code»).

Hook	Quando	Cosa fa
recinto	prima di ogni scrittura, anche via Bash, e prima di `git commit` e `git push`	**Blocca**
manoscritto	prima di scrivere in `04-manoscritto/`	Oggi **avvisa** se manca `zb avvio`
sessione	all'inizio della sessione	Annota la sessione
consumo	dopo le letture e le chiamate agli agenti	Conta, non blocca

Si accendono solo sui rami dei libri, con `zb hook attiva --ok`. Sul ramo `motore` sono spenti per scelta.

CLAUDE.md

È il foglio delle regole di avvio. Sta nella radice del repository e Claude Code lo legge a ogni sessione. Dev'essere breve (sotto le 30 righe) e senza nomi di libri (motore/editoriale/risparmio-token.md). Cosa dice: capitolo 4.

Rami dei libri

Ogni libro vive su un ramo suo, nella cartella `libri/<slug>/`. Dettagli nel capitolo 5.

3. Il percorso di un libro

Dall'idea a KDP il percorso ha dodici passi. Il motore lo fa rispettare: i gate sono nel codice (motore/editoriale/00-percorso.md; motore/PROCEDURA.md §5).

#	Fase	Chi lavora	Fermata
1	Briefing	autore	—
2	Apertura della cartella unica (`zb nuovo`)	motore	—
3	Struttura	architetto	gate `struttura`

#	Fase	Chi lavora	Fermata
4	Revisione della struttura	editor strutturale, **sessione separata**	nel gate `struttura`
5	Documenti	direttore editoriale	gate `documenti`
6	Pagina campione	scrittore con voce	gate `pagina_campione`
7	Stesura	scrittore con voce	gate `primi_capitoli`
8	Revisione strutturale del manoscritto	editor strutturale	gate `struttura_manoscritto`

#	Fase	Chi lavora	Fermata
9	Line editing	line editor	gate `line`
10	Copyediting	correttore di italiano	gate `copy`
11	Bozze	correttore di bozze	gate `bozze`
12	Chiusura meccanica	motore	—

Dopo il passo 8 c'è un **passo opzionale**: i lettori ciechi e il panel di mercato. Solo con il tuo ok (motore/PROCEDURA.md §5).

Come funziona un gate

Quando il lavoro di una fase è finito, Claude Code lancia `zb pronto`. Il motore apre la fermata e mostra il primo blocco

di circa 120 righe. Le risposte dell'autore (motore/PROCEDURA.md §5):

- **avanti**: il blocco dopo. Non approva nulla.
- **ok**: approva. Vale solo se hai visto tutti i blocchi.
- **ok, lotti da N**: approva e fissa quanti capitoli scrivere di fila.
- **correggi: …**: registra una correzione. La fermata resta aperta.
- **stato**: mostra lo stato, senza cambiare nulla.

Qualunque altra frase è una domanda: non cambia lo stato. Non si passa alla fase dopo senza `ok`. Se un ruolo trova un problema grave, si ferma e lo segnala (motore/editoriale/00-percorso.md).

I gate attivi sono nel campo `gate` di `libro.yaml`. Il gate `struttura` e il gate `controllo_fallito` sono **sempre attivi** (motore/PROCEDURA.md §5).

Prima di ogni sessione: `zb avvio`

A ogni sessione, e dopo ogni compattazione della memoria di Claude Code, si lancia `zb avvio <libro> [--ruolo R]`. Si ferma senza scrivere nulla se qualcosa non va: modifiche non salvate, ramo sbagliato, ramo non allineato a GitHub, un documento approvato cambiato di nascosto, due ruoli nella stessa sessione (motore/PROCEDURA.md §4).

Se tutto va bene stampa la riga «Letto: …», con i file letti, le righe e le impronte. Claude Code la riporta prima di scrivere.

Un ruolo per sessione. Per cambiare ruolo: salvare, aprire una sessione nuova, `zb avvio`. Chi ha scritto un testo non lo revisiona nella stessa sessione (motore/editoriale/00-percorso.md; motore/dati/hook.yaml).

Passi 1-2. La nascita

1. Sul ramo del libro, mai sul ramo `motore`, l'autore compila il modello

`motore/modelli/briefing.md`.
2. Claude Code lancia `zb nuovo <briefing>`.
3. Se il briefing è incompleto, `zb nuovo` elenca al massimo 5 cose mancanti e **non crea nulla**.
4. Se è completo, crea la cartella unica `libri/<slug>/` con `libro.yaml`, `stato.yaml`, `LEGGIMI.md`, `cronologia.yaml`, `nomi_propri.txt`, e la fase **struttura**: dentro `03-struttura/` nascono solo i modelli della struttura. I documenti nascono dopo l'ok.

Per un libro che ha già dei documenti c'è `zb adotta`: prepara delle bozze con la fonte di ogni valore (motore/PROCEDURA.md §3 e §3b; motore/passaggio-di-consegne.md).

Passi 3-4. Struttura e sua revisione

L'architetto scrive nove file in `03-struttura/`: premessa (con il contratto della storia), logline e tema, personaggi

chiave, atti e svolte, scaletta, sottotrame, tensione e ritmo, piano parole, scelte fisse. Poi l'editor strutturale la giudica **in una sessione separata**, in `06-diagnostica/revisione-struttura.md` (motore/editoriale/struttura/01-fase-struttura.md; motore/PROCEDURA.md §5).

Il gate `struttura` è sempre attivo. Finché non dici `ok`, non c'è nessun documento e nessun capitolo.

Passi 5-6. Documenti e pagina campione

Fase	Lavoro	Fermata
documenti	bibbia, cronologia, nota di direzione, manuale di stile	gate `documenti`
pagina campione	`05-revisioni/pagina-campione.md`	gate `pagina_campione`

(motore/PROCEDURA.md §5)

Passo 7. La stesura

Per ogni capitolo, l'agente scrittore (chiamato dalla sessione principale, mai due capitoli in parallelo) (motore/PROCEDURA.md §6):

1. rilegge `LEGGIMI`, manuale, scheda della voce, scaletta, piano parole, cronologia, **le capsule** dei capitoli precedenti e solo l'ultima pagina del precedente;
2. riporta la riga «Letto»;
3. scrive `04-manoscritto/NN-<slug>.md`;
4. `zb esito <libro> N`: parole, stile, continuità, ripetizioni vicine e **audit anti-IA** (report in `06-diagnostica/capitoli/` e `06-diagnostica/anti-ia/`);
5. `zb capsula <libro> N`: l'agente capsula scrive `07-capsule/NN.md`, di nuovo `zb capsula` per controllarla;
6. salva: commit e push.

Se un controllo fallisce: **una sola** correzione mirata, poi di nuovo `zb esito`. Al secondo fallimento di fila si apre il gate `controllo_fallito` e decidi tu.

Dopo il capitolo 3 c'è la fermata `primi_capitoli`. Con l'ok scegli i lotti; poi i capitoli si scrivono di fila. Scritte tutte le unità del piano, la fase passa a «revisione_manoscritto».

Passo 8. Revisione strutturale del manoscritto

L'editor strutturale (sessione separata) legge struttura e capsule, e solo dove serve i capitoli segnalati. Scrive `06-diagnostica/revisione-manoscritto.md`. Gate `struttura_manoscritto`. Dopo l'ok, gli interventi si applicano con modifiche puntuali; poi `zb esito` sui capitoli toccati (motore/PROCEDURA.md §5).

Passo opzionale: lettori ciechi. Con il tuo ok, l'agente `lettore-cieco` legge un campione e riferisce l'esperienza di

lettura (motore/editoriale/agenti/lettore-cieco.md).

Passi 9-11. Line, copy, bozze

Fase	Chi	Report	Gate
line	line editor, per capitolo (max 3 in parallelo)	`06-diagnostica/line/NN.md`	`line`
copy	correttore di italiano, per capitolo	`06-diagnostica/copy/NN.md`	`copy`

Fase	Chi	Report	Gate
bozze	correttore di bozze, a testo fermo (`compila`, `impagina`, `pdf`)	`06-diagnostica/bozze.md`	`bozze`

Dopo l'`ok` a `line` e `copy` si applicano gli interventi approvati, con modifiche puntuali: l'hook rifiuta la riscrittura intera di un capitolo da parte di un revisore (motore/PROCEDURA.md §5 e §11).

Passo 12. La chiusura meccanica

L'ordine è fisso (motore/PROCEDURA.md §7): continuita, ortografia, compila, impagina, pdf, pacchetto, kdp, pronto. `pacchetto` va prima di `kdp`: il controllo KDP legge la scheda del pacchetto.

Chiusura 1: `zb continuita`

- **Cosa fa.** Controlla date, giorni della settimana, età dei personaggi, durate, cifre, nomi scritti in due modi, ordine delle date.
- **Cosa serve prima.** I capitoli in `04-manoscritto/` e il file `cronologia.yaml` del libro.
- **Cosa produce.** Il report `06-diagnostica/continuita.md`, con una lista di controllo per capitolo.

(motore/script/continuita.py; motore/PROCEDURA.md §7)

Chiusura 2: `zb ortografia`

- **Cosa fa.** Tre livelli: (a) Hunspell, un dizionario italiano automatico, sempre; (b) LanguageTool, un correttore più fine, solo se installato; (c) una lista di controllo per la lettura umana, sempre.
- **Cosa serve prima.** Hunspell con il dizionario italiano; le parole ammesse del libro (`nomi_propri.txt`). Per il livello (b), LanguageTool e la variabile `ZB_LANGUAGETOOL`.

- **Cosa produce.** Il report `06-diagnostica/controllo-ortografico.md`. Dice quali livelli sono stati eseguiti.

(motore/script/ortografia.py; motore/PROCEDURA.md §7)

Chiusura 3: `zb compila`

- **Cosa fa.** Mette insieme il libro: frontespizio, pagina del copyright, sommario, parti, capitoli in ordine, appendici. Il sommario lo genera sempre il motore.
- **Cosa serve prima.** `libro.yaml` e i capitoli. Le appendici dichiarate in `libro.yaml` e presenti in `05-appendici/`.
- **Cosa produce.** `05-output/<titolo>-completo.md`. Le immagini delle appendici vanno in `05-output/appendici/`.

(motore/script/compila.py; motore/PROCEDURA.md §7)

Chiusura 4: `zb impagina`

- **Cosa fa.** Fa il PDF di stampa con Chromium (un navigatore web usato come tipografo). Margini dalla tabella KDP, pagine iniziali senza numero, sommario con le pagine vere, formato esatto della pagina, metadati Title e Author.
- **Cosa serve prima.** Il file compilato del passo 3; nel computer di lavoro Chromium, PyMuPDF e markdown.
- **Cosa produce.** `05-output/<titolo>.pdf`.

(motore/script/impagina.py; motore/README.md, «Ambiente»)

Chiusura 5: `zb pdf`

- **Cosa fa.** Verifica il PDF: numero di pagine; Title e Author uguali a `libro.yaml`; font incorporati; margini contro i minimi KDP; ogni voce del sommario sulla pagina giusta.
- **Cosa serve prima.** Il PDF del passo 4; gli strumenti `pdfinfo`, `pdffonts`, `pdftotext`.

- **Cosa produce.** Il report `06-diagnostica/verifica-pdf.md`.

(motore/script/verifica_pdf.py)

Chiusura 6: `zb pacchetto`

- **Cosa fa.** Prepara la cartella per KDP con modelli già compilati con i dati di `libro.yaml`. **Non** scrive testi commerciali (descrizione, quarta, parole chiave, categorie): li scrive Claude Code dopo. Non sovrascrive mai un file che c'è già.
- **Cosa serve prima.** `libro.yaml`.
- **Cosa produce.** In `06-pubblicazione/`: `scheda-amazon.md`, `quarta.md`, `brief-copertina.md`, `conferme-autore.yaml`, `promemoria-ai.md`, `checklist.md`.

(motore/script/pacchetto.py)

Chiusura 7: `zb kdp`

- **Cosa fa.** Controlla la conformità a KDP in 15 sezioni: titolo, sottotitolo, serie, descrizione, autore, editore, parole chiave, categorie, copertina, ISBN, cartaceo, eBook, dichiarazione sull'IA,

contenuti, recensioni. Ciò che deve confermare l'autore, finché non è confermato, risulta KO.

- **Cosa serve prima.** `libro.yaml`, la scheda del passo 6, il file compilato, il PDF, `conferme-autore.yaml`, i dati KDP del motore; l'eBook `ebook.docx` se il libro lo prevede.
- **Cosa produce.** `06-pubblicazione/conformita-kdp.md`, una riga OK/KO per sezione.

(motore/script/conformita_kdp.py)

Chiusura 8: `zb pronto`

- **Cosa fa.** Controlla che ci siano tutti i risultati dei passi 1-7. Se sì, il libro passa a «chiuso».
- **Cosa serve prima.** I sette file: `continuita.md`, `controllo-ortografico.md`, il compilato, il PDF, `verifica-pdf.md`, `checklist.md`, `conformita-kdp.md`.
- **Cosa produce.** Il cambio di fase in `stato.yaml`. Se manca qualcosa,

elenca cosa e con quale comando farlo.

(motore/script/fase.py, elenco CHIUSURA e cmd_pronto)

> **Prima di pubblicare.** I dati KDP del motore non sono mai stati riverificati dall'autore sulle pagine ufficiali (`ultima_verifica: null`). La verifica si registra con `zb verifica KDP fatta` (motore/report-motore.md §11; motore/passaggio-di-consegne.md, «Punti aperti» 1).

Dopo ogni passo: il salvataggio

Dopo ogni passo che scrive: report in `06-diagnostica/`, commit, push (con `zb push`), controllo dello stato. Se il push fallisce, ci si ferma e si riporta l'errore. Mai dire «fatto» se il lavoro non è su GitHub (CLAUDE.md; motore/PROCEDURA.md §8).

4. Le regole

Cosa dice CLAUDE.md

In parole semplici dice (CLAUDE.md):

1. **Regola madre.** Il motore resta pulito; il ramo `motore` non contiene mai un libro.
2. **Briefing nuovo:** prima azione `zb nuovo`. Tutto ciò che riguarda il libro vive solo nella sua cartella; il suo nome non compare altrove.
3. **Recinto.** L'hook blocca le scritture fuori dal libro attivo. Solo se scrivi «lavoro sul motore», Claude Code lancia `zb lavoro motore`.
4. **Letture.** Per lavorare su un libro: `motore/PROCEDURA.md` e il `LEGGIMI.md` del libro. Se manca il percorso, Claude Code lo chiede prima di toccare file.
5. **Avvio.** Prima di scrivere: `zb avvio <libro> [--ruolo R]` e la riga «Letto». Se si ferma, Claude Code si ferma e dice perché. **Un ruolo per sessione.**

6. **Cosa legge il motore:** solo `motore/`, la cartella del libro e la banca-voci (sola lettura). `docs/` non si legge, se non lo chiedi tu.
7. **Orchestrazione.** La sessione principale non scrive prosa. `zb agenti prossimo <libro>` dice quale agente chiamare. Capitoli sempre in sequenza; line, copy e capsule in parallelo, al massimo come in `motore/dati/orchestrazione.yaml`.
8. **Report di sessione** ogni 5 scambi, o quando scrivi «passaggio». Va nella cartella del libro (`passaggio-di-consegne.md`); quello del motore solo per il lavoro sul motore. Sette sezioni fisse: Stato; Decisioni prese; Regole da non violare; Fatto in questa sessione; Punti aperti; Prossimo passo; Ripartenza.
9. **Precedenza:** una sola scala, in `motore/editoriale/LEGGIMI.md`.
0. **Metodo:** proponi, l'autore approva, applica. Mai approvare da solo.
1. **Risposte dell'autore:** ok, avanti, correggi, stato.

2. **Ramo:** quello della sessione, letto da git. **Tono:** il testo precedente è un registro dei fatti.
3. **Salvataggio** a ogni passo: add, commit, push (`zb push`), controllo. Resoconto con codice, file e «branch allineato a origin: sì/no».

Gli hook

Il recinto: blocca

- **Cosa guarda.** Ogni scrittura di file (Write, Edit, MultiEdit, NotebookEdit) e anche i comandi di shell (**Bash**): redirezioni, `tee`, `sed -i`, `cp`, `mv`, `rm`.
- **Regola.** Si può scrivere solo in `libri/<slug-attivo>/`. Il resto è rifiutato con il messaggio «RECINTO: …».
- **Prima di `git commit` e `git push`** controlla la cartella unica: un commit che tocca il libro e altro è bloccato.
- **Revisori.** Un revisore non può riscrivere un capitolo per intero: solo modifiche puntuali.

- **Libro attivo.** Lo scrivono `zb nuovo` e `zb lavoro libro <slug>`; sta nella cartella git, non in un file tracciato.
- **Lavoro sul motore.** Con «lavoro sul motore», Claude Code lancia `zb lavoro motore`: si può scrivere nel motore, ma non in `libri/`.

(motore/PROCEDURA.md §10-11; motore/README.md, «Hook di Claude Code»)

Limiti dichiarati. Su Bash il recinto non vede tutto: script che scrivono senza nominare il percorso, variabili di comandi precedenti, `$(…)`, `awk -i`, `find -delete`, `xargs`. Restano i controlli su git di `zb avvio` e `zb push` (motore/report-motore.md §11, punto 3).

Il manoscritto: avvisa

Se si scrive in `04-manoscritto/` senza aver fatto `zb avvio` in quella sessione, oggi stampa «ATTENZIONE: …» e lascia passare. Lo annota in `.zb/avvisi-hook.log` del libro. La modalità sta in `motore/dati/hook.yaml`: oggi `avviso`. Il

passaggio a `blocco` sarà un commit a parte, dopo il tuo ok (motore/PROCEDURA.md §11).

La separazione dei ruoli: blocco

Chi ha scritto un testo non lo revisiona nella stessa sessione. Il recinto annota chi scrive in `.zb/scritture.log`. In `motore/dati/hook.yaml` la scelta attuale è `separazione_ruoli: blocco`: il motore **si ferma** al `zb pronto` del gate e all'avvio del ruolo di revisione. Puoi tornare ad `avviso` con un commit a parte (motore/dati/hook.yaml).

Il consumo e la sessione

L'hook di sessione scrive il codice della sessione; dopo una compattazione, i vecchi «avvio fatto» non valgono più. L'hook di consumo conta file letti, righe e agenti chiamati, per `zb consumo`. Misura le letture, non i token (motore/PROCEDURA.md §11; motore/report-motore.md §11).

Come si accendono

`zb hook attiva --ok`, solo sui rami dei libri e solo dopo il tuo ok. Crea `.claude/settings.json`. `zb hook stato` dice se sono accesi; `zb hook disattiva` li spegne. Gli hook sono «una protezione in più, non una garanzia» (motore/PROCEDURA.md §11).

La cartella unica

Tutto ciò che riguarda un libro vive in `libri/<slug>/`. `zb cartella <libro>`, e da soli `zb avvio` e `zb push`, controllano su git: file modificati fuori dal libro, commit «misti», titolo del libro in file fuori dalla cartella (motore/PROCEDURA.md §10).

Le eccezioni sono dichiarate in `motore/dati/cartella-unica.yaml`: `CLAUDE.md`, `.gitignore`, `.claude/settings.json`, `.claude/agents/` e i merge del motore. I commit misti già fatti li dichiari tu, con il motivo: la storia non si riscrive.

`zb push` e le prove sul motore

`zb push` fa il push del ramo corrente; se fallisce riprova, poi tenta un ramo alternativo, poi un `git bundle`. Prima controlla la cartella unica. Sul **lavoro del motore** vuole `zb prove` intera e tutta OK sullo stato attuale (zb aiuto; motore/passaggio-di-consegne.md, «Decisioni prese»). Oggi le prove sono 520 su 520 OK.

Altre protezioni, senza hook

Chi	Cosa fa
Codice del motore	Ogni comando scrive solo nella cartella del libro
`zb nuovo`	Rifiuta un libro sul ramo `motore` o fuori da `libri/<slug>/`
`zb recinto`	Prova che i comandi non scrivano fuori dal libro

Chi	Cosa fa
`zb separazione`	Prova che in `motore/` non ci siano pezzi di libri veri

La precedenza

Quando due regole si contraddicono, vince quella più in alto. Una sola scala, quella del capitolo 2: libro e `libro.yaml`, poi briefing e voce, poi profilo di genere, poi ruoli, poi procedura. **Ogni contrasto si segnala** (motore/editoriale/LEGGIMI.md; CLAUDE.md).

5. I rami

Il ramo `motore`

- Contiene `motore/`, `CLAUDE.md`, `.gitignore` e la cartella `docs/` con i documenti di lettura.
- Mai un libro, mai una cartella `libri/`.
- È l'unico ramo dove si lavora sul motore.
- Qui gli hook sono spenti: non c'è `.claude/settings.json`, e una prova lo richiede.
- Qui `zb nuovo` e `zb lavoro libro` si rifiutano.

(motore/passaggio-di-consegne.md, «Stato» e «Decisioni prese»; motore/README.md)

I rami dei libri

- Un libro per ramo, nella cartella `libri/<slug>/`.
- Ogni ramo porta una copia di `motore/`. Il motore aggiornato arriva con un

merge dal ramo `motore`, mai con un rebase.
- Con gli hook e gli agenti accesi ci sono anche `.claude/settings.json` e `.claude/agents/`.
- Ogni ramo di libro ha il suo `CLAUDE.md`, per tua scelta. Non si toccano da qui.

I nomi dei rami dei libri non si scrivono sul ramo `motore`: lo vieta la regola sui nomi reali (motore/passaggio-di-consegne.md, «Regole da non violare»).

I rami d'archivio

Ramo	Cosa contiene
`archivio/motore-ritirati`	Lo script `pulizia.py`, ritirato, con le sue prove e gli attesi
`archivio/motore-docs`	Le esportazioni precedenti di `docs/`, con la banca-voci integrale

Ramo	Cosa contiene
`archivio/ghimoney`	Un progetto separato tolto dal motore. Solo archivio

(motore/passaggio-di-consegne.md, «Stato»)

I rami di sessione (`ccr-*`, `claude/*`) sono lavori di sessione non legati al motore.

I libri vecchi: una scelta, non un errore

I rami dei libri vecchi sono copie di lavoro per la riscrittura. Hanno un `CLAUDE.md` proprio **per tua scelta** e non vanno riallineati al motore (motore/passaggio-di-consegne.md, «Regole da non violare»). Su un ramo vecchio comanda il suo `CLAUDE.md`. Un aggiornamento del motore arriva a un ramo di libro solo con un merge, ramo per ramo.

6. Cosa non esiste

Skill e comandi slash: restano non usati

Non c'è `.claude/skills/`, non c'è `.claude/commands/`, non c'è un `SKILL.md` del sistema. I «comandi» del sistema sono i comandi `zb`. Il livello editoriale **sostituisce ogni skill citata ma assente**: la sua prima regola è «una fonte di verità» (motore/editoriale/LEGGIMI.md, «Regole», punto 1).

Anti-IA: ora esiste

Una sezione anti-scrittura-da-IA c'è, in `motore/editoriale/anti-ia/` (motore/editoriale/LEGGIMI.md; motore/PROCEDURA.md §6):

- `catalogo-it.md`: gli schemi da IA in italiano (ritmo, formule vuote, spiegare ciò che si è mostrato, dialogo…) e le regole di ritmo. Lo usano scrittore e line editor;

- `catalogo-it-esteso.md` e `formule.yaml`: altri schemi e 37 espressioni misurate da `zb esito`;
- l'**audit**: `zb esito` misura monotonia delle frasi, aperture uguali, formule per mille parole, chiusure uguali, ripetizioni ravvicinate. Sono avvisi, con riga e proposta; il motore non riscrive. Con `anti_ia: {modalita: blocco}` in `libro.yaml` diventano KO.

Correttore di italiano: ora esiste

Esistono il ruolo 6 e i criteri in `motore/editoriale/correttore/criteri-correttore.md` (accenti e apostrofi, accordi, tempi e modi, punteggiatura, coppie di parole…). Il ruolo lavora insieme a `zb ortografia`, a tre livelli: Hunspell, LanguageTool se installato, lista per la lettura umana (motore/PROCEDURA.md §7). LanguageTool si installa con `zb ortografia installa`, fuori dal repository, e si indica con `ZB_LANGUAGETOOL`.

Cosa davvero non c'è

- La **griglia a cinque assi** e la **lettura a freddo**: il repository da cui dovevano venire non era accessibile (motore/passaggio-di-consegne.md, «Punti aperti» 4).
- Un controllo automatico che lanci le prove a ogni modifica: si lanciano a mano con `zb prove`.
- Un ambiente che resti da una sessione all'altra: in un container nuovo si reinstallano i requisiti (motore/report-motore.md §11, punto 7).
- I nomi dei libri reali in `motore/dati/nomi_vietati.txt`: il file è vuoto.

7. Appendici e indice

Come stanno oggi

> Oggi il motore **include** le appendici. Lo fa dal commit `71a36cc` («Motore: appendici dichiarate in libro.yaml e sommario completo», 14 ottobre 2026). Le regole sono in `motore/PROCEDURA.md` §7 (storia git del ramo motore; motore/script/compila.py).

Appendici. Sono i materiali dopo l'ultima unità del libro, epilogo compreso: soluzioni di enigmi, mappe, glossari, note (motore/PROCEDURA.md §7).

- Stanno nella cartella `05-appendici/` del libro (motore/dati/struttura-libro.yaml).
- Si dichiarano in `libro.yaml`, campo `appendici`, nell'ordine di lettura: titolo, file, didascalia per le immagini (motore/PROCEDURA.md §7; motore/dati/libro.schema.yaml).

- Formati: testo `.md`; immagini `.svg`, `.png`, `.jpg`. Un `.svg` diventa PNG con Chromium (motore/PROCEDURA.md §7).
- Ogni appendice comincia su una pagina nuova (motore/PROCEDURA.md §7).
- Un file dichiarato e assente ferma `zb compila`. Un file presente ma non dichiarato dà solo un avviso e resta fuori (motore/script/compila.py).
- Le appendici non vanno mai in `04-manoscritto/` (motore/PROCEDURA.md §7).
- Un'appendice non anticipa nulla: le soluzioni stanno solo in appendice (motore/PROCEDURA.md §7).
- Un libro senza il campo `appendici` si compila come prima (motore/PROCEDURA.md §7).

Indice (sommario). Lo genera `zb compila`, sempre. Non si scrive a mano (motore/PROCEDURA.md §7).

- Contiene, in ordine: parti, prologo, capitoli («N. Titolo»), interludi, epilogo, appendici (motore/PROCEDURA.md §7).
- `zb impagina` aggiunge la pagina vera di ogni voce; `zb pdf` controlla che sia giusta (motore/PROCEDURA.md §7).
- Per cambiare una voce si cambia la sua fonte (titolo del capitolo, `libro.yaml`), poi si rilancia `zb compila` (motore/PROCEDURA.md §7).

Cosa è stato deciso

Decisione	Stato nel repository
Cartella appendici nel libro	Fatto: `05-appendici/`
Indice sempre presente	Fatto: lo genera `zb compila`
Motore generico	Fatto: le regole valgono per ogni libro; nessun libro nel motore

(motore/dati/struttura-libro.yaml;
motore/PROCEDURA.md §7;
motore/README.md; CLAUDE.md)

8. I 17 punti deboli

Sono i limiti noti di `motore/report-motore.md` (§11), i punti aperti di `motore/passaggio-di-consegne.md` e le scelte in attesa di `docs/DA_VERIFICARE_AUTORE.md`. Ordine: dal più serio al più lieve. È una valutazione di questa guida.

Importanti: prima di pubblicare o fidarsi

1. I dati KDP non sono mai stati verificati da te. In `motore/dati/kdp.yaml` c'è `ultima_verifica: null`. Vanno controllati sulle pagine ufficiali e registrati con `zb verifica KDP fatta`. `zb kdp` lo segnala.

2. Le soglie anti-IA sono provvisorie. Sono tarate solo sui due mini-libri di prova e sono uguali per ogni genere. Sono avvisi, non blocchi. Vanno riviste su testi veri, genere per genere.

3. Gli hook sono una protezione in più, non una garanzia. Su Bash il recinto non vede alcune scritture (capitolo 4). Restano i controlli su git di `zb avvio` e `zb push`.

4. La separazione dei ruoli distingue le sessioni solo con l'hook di sessione (o con `ZB_SESSIONE`). Con la sessione «locale» dà solo un avviso. I log in `.zb/` stanno solo nel container.

5. Il lavoro si perde se non è su origin. L'ambiente non è persistente: in un container nuovo vanno reinstallati requisiti, Hunspell e LanguageTool.

Medi: misure e controlli incompleti

6. Il consumo misura le letture, non i token. Nella prova sul mini-libro, per lo scrittore del capitolo 3: 2444 parole lette in 4 file senza capsule, 796 parole in 5 file con le capsule.

7. La continuità ha limiti. I tempi relativi si controllano solo se nessuna

data nota coincide; le differenze d'età solo per i personaggi con data di nascita completa in `cronologia.yaml`.

8. `nomi_vietati.txt` è vuoto. Il controllo di separazione non trova da solo i nomi dei libri reali. La cartella unica controlla il titolo del libro attivo sul suo ramo.

9. PDF: TrimBox e BleedBox non sono scritti. Trasparenze e immagini non sono controllate.

10. I mini-libri sotto le 24 pagine hanno la sezione 11 della conformità KDP in KO per costruzione.

11. LanguageTool: il ramo di scarico da Maven Central è stato provato prima dell'errore 429 del 15 ottobre 2026. Dopo, solo la cache locale.

Scelte tue ancora da confermare

12. Separazione dei ruoli in blocco e capsule obbligatorie. Sono scelte del

motore: `separazione_ruoli: blocco`, `capsule: obbligatorie` per i libri nuovi (nei vecchi, solo un avviso). Se preferisci, si torna ad avviso.

13. Ripetizioni ravvicinate. Finestra di 60 parole piene, 3 occorrenze, al massimo 12 gruppi ogni mille parole. È un algoritmo nostro: il repository indicato non lo conteneva.

14. Due dubbi nei testi. Nei criteri del correttore la coppia «avvenire/avvenire» è ripetuta: forse un refuso. Nel modello della scaletta si scrive «promesse aperte / chiuse», il glossario dice «pagate».

15. Il modello leggero non è provato sul copyediting. Il copyediting usa già il modello medio; il leggero fa solo le capsule.

Lievi

16. La storia git contiene ancora i documenti spostati sui rami d'archivio: non si riscrive.

17. Cose non prese. La griglia a cinque assi e la lettura a freddo (repository non accessibile).

(motore/report-motore.md §11; motore/passaggio-di-consegne.md, «Punti aperti»; docs/DA_VERIFICARE_AUTORE.md)

9. Dove trovare le cose

Ramo «motore» vuol dire anche: nella copia del motore di ogni ramo di libro aggiornato.

Cosa cerchi	File	Ramo
Regole di avvio	`CLAUDE.md`	motore (ogni ramo di libro ha il suo)
Procedura completa	`motore/PROCEDURA.md`	motore
Percorso e fermate	`motore/editoriale/00-percorso.md`	motore
Precedenza (una scala) e regole editoriali	`motore/editoriale/LEGGIMI.md`	motore
I sette ruoli	`motore/editoriale/ruoli/`	motore

Cosa cerchi	File	Ramo
Gli agenti	`motore/editoriale/agenti/`	motore
Modelli degli agenti	`motore/dati/modelli-agenti.yaml`	motore
Orchestrazione e parallelo	`motore/dati/orchestrazione.yaml`	motore
Fase struttura e suoi modelli	`motore/editoriale/struttura/`	motore
Anti-IA	`motore/editoriale/anti-ia/`	motore
Criteri del correttore	`motore/editoriale/correttore/criteri-correttore.md`	motore
Glossario	`motore/editoriale/glossario.md`	motore
Risparmio di token	`motore/editoriale/risparmio-token.md`	motore

Cosa cerchi	File	Ramo
Elenco dei comandi	`python3 motore/zb aiuto`	motore
Regole d'uso e hook	`motore/README.md`	motore
Stato del lavoro sul motore	`motore/passaggio-di-consegne.md`	motore
Limiti noti del motore	`motore/report-motore.md` §11	motore
Scelte da confermare	`docs/DA_VERIFICARE_AUTORE.md`	motore
Analisi dei repository esterni	`docs/ANALISI_REPOSITORY.md`	motore
Progetto originale	`motore/proposta-motore.md`	motore
Profili di genere	`motore/profili/`	motore
Modelli vuoti	`motore/modelli/`	motore

Cosa cerchi	File	Ramo
Dati KDP	`motore/dati/kdp.yaml`	motore
Lista nera, parole ammesse	`motore/dati/lista-nera.yaml`, `parole-ammesse-it.txt`	motore
Cartelle di un libro, campi di `libro.yaml`	`motore/dati/struttura-libro.yaml`, `libro.schema.yaml`	motore
Hook: script e modalità	`motore/script/hook_*.py`, `motore/dati/hook.yaml`	motore
Cartella unica: eccezioni	`motore/dati/cartella-unica.yaml`	motore
Output compatto	`motore/dati/output.yaml`	motore

Cosa cerchi	File	Ramo
Hook e agenti accesi	`.claude/settings.json`, `.claude/agents/`	ramo del libro
Impaginazione e font	`motore/stampa/`	motore
Prove e mini-libri inventati	`motore/script/prove.py`, `motore/prove/`	motore
Un libro	`libri/<slug>/`	ramo del libro
Struttura del libro	`libri/<slug>/03-struttura/`	ramo del libro
Bibbia del libro	`libri/<slug>/02-bibbia/bibbia.md`	ramo del libro
Capitoli	`libri/<slug>/04-manoscritto/`	ramo del libro
Capsule	`libri/<slug>/07-capsule/`	ramo del libro

Cosa cerchi	File	Ramo
Appendici	`libri/<slug>/05-appendici/`	ramo del libro
PDF e libro compilato	`libri/<slug>/05-output/`	ramo del libro
Report dei controlli e dei ruoli	`libri/<slug>/06-diagnostica/`	ramo del libro
Pacchetto KDP	`libri/<slug>/06-pubblicazione/`	ramo del libro
Report di sessione del libro	`libri/<slug>/passaggio-di-consegne.md`	ramo del libro
Voci narranti	`indice.yaml`, `voci/<slug>/`	banca-voci (repository a parte)
Esportazioni precedenti di `docs/`	ramo `archivio/motore-docs`	archivio
Questa guida	`docs/GUIDA_AL_SISTEMA.pdf` e `.md`	motore

(motore/README.md; motore/editoriale/; motore/dati/struttura-libro.yaml)

Fine della guida.